

**Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»**

**Навчально-науковий інститут атомної і теплової енергетики  
Кафедра цифрових технологій в енергетиці**

**ЗВІТ**

**з розрахунково-графічної роботи**

**з дисципліни «Візуалізація графічної та геометричної інформації»**

**на тему: «Операції над текстурними координатами»**

**Варіант № 1**

**Виконав:**  
студент групи ТР-51мп  
Авраменко Олег Вячеславович

**Перевірив:**  
доцент кафедри, к.т.н.,  
Демчишин Анатолій Анатолійович

**Дата здачі \_26.12.2025**

**Київ – 2025**

## 1. Опис завдання

Метою даного завдання є розширення WebGL-застосунку, розробленого в попередніх роботах (РА1 - РА3), для візуалізації параметричної поверхні обертання з підтримкою освітлення та текстуровування.

Завдання для РГР (мій варіант 1 – непарний) додає такі вимоги:

1. Повторно використати систему текстуровування з РА3 (дифузна, specular та normal-текстури).
2. Реалізувати масштабування текстурних координат (texture coordinates scaling) відносно точки в просторі  $(u,v)$  – для непарних варіантів виконується масштабування, для парних – обертання.
3. Надати можливість пересувати точку по поверхні в UV-просторі за допомогою клавіатури: клавіші A / D рухають точку вздовж параметра  $u$ ; клавіші W / S рухають точку вздовж параметра  $v$ .
4. Масштабування текстури завжди здійснюється відносно поточного положення точки.

## 2. Теоретичні відомості

У даному завданні використовується поверхня обертання модифікованої параболи. Базова крива задається у вертикальній площині, після чого обертається навколо осі  $z$  під кутом  $u$ .

Нехай:

$t$  – параметр уздовж твірної кривої,

$u$  – кут обертання.

Поверхня задається за формулами 2.1-2.4:

$$R(t) = a + t * \cos(\theta) + c * t^2 * \sin(\theta), \quad (2.1)$$

$$x(u,t) = R(t) * \cos(u), \quad (2.2)$$

$$y(u,t) = R(t) * \sin(u), (2.3)$$

$$z(t) = t * \sin(\theta) + c * t^2 * \cos(\theta), (2.4)$$

де  $a, c$  і  $\theta$  – параметри, що задаються користувачем.

Дискретизація відрізків  $u$   $[0, 2\pi]$  та  $t$   $[t_{\min}, t_{\max}]$  регулярною сіткою дає набір вершин трикутної сітки. Сусідні точки на цій сітці з'єднуються в трикутники за допомогою індексного буфера, що дозволяє ефективно відображати поверхню в WebGL.

Для коректного освітлення та normal mapping потрібна локальна система координат на поверхні.

Для параметричної поверхні  $p(u,t)$  дві дотичні можна отримати як часткові похідні за формулами 2.5-2.6:

$$p_u = \frac{\partial p}{\partial u}, (2.5) \quad p_t = \frac{\partial p}{\partial t}. (2.6)$$

Як дотичний вектор  $T$  використовується аналітична дотична вздовж параметра обертання  $u$ .

Аналітична нормаль  $N$  обчислюється за формулами  $p_u$  x  $p_t$ , а потім нормалізується.

Дотична вздовж  $u$  рахується за допомогою формули 2.7:

$$T(u,t) = (-R(t) * \sin(u), R(t) * \cos(u), 0), (2.7)$$

після чого вектор нормалізується до одиничної довжини.

Аналітична нормаль також нормується. Надалі ці два вектори використовуються для побудови базису TBN у вершинному шейдері.

Normal mapping дозволяє імітувати дрібні нерівності поверхні без збільшення геометричної складності. Замість зміщення вершин ми змінюємо нормаль, яка береться для розрахунку освітлення у кожному фрагменті.

Normal map зберігає вектор нормалі  $n_{\text{tan}}$  у тангенціальному просторі. Оскільки значення текстури зберігаються у діапазоні  $[0,1]$  для перетворення в діапазон  $[-1,1]$  використовується формула 2.8:

$$n_{\text{tan}} = 2 * c_{\text{RGB}} - 1 \quad (2.8)$$

Щоб використати цю нормаль в рівняннях освітлення, потрібно перетворити її з тангенціального простору в простір, де задані вектори світла та огляду. Для цього будується матриця TBN 2.9:

$$\text{TBN} = [\text{T} \ \text{B} \ \text{N}], \quad (2.9)$$

де:

- $\text{T}$  – дотична,
- $\text{N}$  – нормаль,
- $\text{B} = \text{N} * \text{T}$  – бінормаль

Тоді нормаль у просторі споглядача розраховується формулою 2.10:

$$N_{\text{eye}} = \text{TBN} * n_{\text{tan}} \quad (2.10)$$

Таким чином отримуємо нормалі, які значно точніше описують локальну геометрію, ніж просто інтерпольовані вершинні нормалі.

Класична модель освітлення Фонга має вигляд формули 2.11:

$$I_{\text{rgb}} = I_{\text{ambient}} + I_{\text{diffuse}} + I_{\text{specular}} \quad (2.11)$$

Де складові формули (2.12-2.14):

- Амбієнтна складова:  $I_{\text{ambient}} = k_a * C_{\text{diffuse}}$  (2.12)
- Дифузна складова:  $I_{\text{diffuse}} = k_d * C_{\text{diffuse}} * \max(N * L, 0)$  (2.13)
- Дзеркальна складова:  $I_{\text{specular}} = k_s * C_{\text{specular}} * (\max(R * V, 0))^\alpha$  (2.14)

Тут:

- $L$  – нормований напрямок на джерело світла у просторі оглядача;
- $V$  – наблизений напрямок на спостерігача (у завданні зафіксований як  $[0,0,1]$ )
- $R$  – відбитий вектор від  $-L$  відносно  $N$ .
- $C_{\text{diffuse}}$  – колір дифузної текстури
- $C_{\text{specular}}$  – колір зі specular-текстури
- $k_a, k_d, k_s$  – глобальні коефіцієнти амбієнтної, дифузної та дзеркальної складових;
- $\alpha$  – показник ступеня блиску (shininess).

Джерело світла рухається навколо поверхні. На кожному кадрі його позиція оновлюється, перетворюється в простір оглядача та передається у шейдер.

Початково текстурні координати задаються формулами 2.15-2.16:

$$S = \frac{u}{2\pi}, \quad (2.15) \quad W = \frac{t - t_{\min}}{t_{\max} - t_{\min}} \quad (2.16)$$

Щоб масштабувати текстуру відносно точки  $P = (u_p, v_p)$  в  $(s, w)$  просторі, застосовується афінне перетворення:

Переносимо координати в систему з центром у півоті за формулою 2.17:

$$c = \begin{pmatrix} s \\ w \end{pmatrix} - \begin{pmatrix} u_p \\ v_p \end{pmatrix} \quad (2.17)$$

Масштабуємо за формулою 2.18:

$$c' = \lambda * c \quad (2.18)$$

де  $\lambda > 1$  – збільшення текстури, а  $\lambda < 1$  – зменшення.

Повертаємося до глобальних координат за формулою 2.19:

$$\begin{pmatrix} s' \\ w' \end{pmatrix} = c' + \begin{pmatrix} u_p \\ v_p \end{pmatrix} \quad (2.19)$$

Таким чином, масштабування прив'язане до точки, яку користувач переміщує по поверхні клавішами W/A/S/D.

### 3. Деталі реалізації

Загальна структура застосунку написана на JavaScript з використанням WebGL і складається з таких основних частин:

- index.html – головна сторінка програми на якій розташовано canvas та панель керування.
- main.js – ініціалізація WebGL, генерація фігур, налаштування шейдерів, текстур, обробка клавіатури і основний цикл візуалізації.
- shader.gp – код вершинного та фрагментного шейдерів (Фонг + normal mapping + масштабування UV).
- Допоміжні скрипти: trackball-rotator (обертання сцени мишею) та m4.js (матричні операції).

Основні глобальні змінні:

- gl – контекст WebGL;
- surface – об'єкт класу Model;

- `shProgram` – допоміжний шар шейдерної програми з усіма локаціями атрибутів та юніформів;
- `spaceball` – об'єкт для інтерактивного обертання сцени мишею;
- `diffuseTexture`, `specularTexture`, `normalTexture` – об'єкти текстур.

Геометрія поверхні обертання генерується за допомогою функції `CreateSurfaceData()`. Вона проходить подвійним циклом по параметрах  $t$  та  $u$ , обчислює просторові координати точки поверхні, а також аналітичні нормалі та тангенти. Радіус обертання визначається формулою, яка включає параметри  $a$ ,  $c$  та кут нахилу  $\theta$ , тому зміна повзунків на панелі керування призводить до миттєвої перебудови всієї сітки. На основі цих розрахунків формуються масиви вершин, нормалей, дотичних векторів і текстурних координат.

Усі масиви передаються в WebGL-буфери в межах класу `Model`. Клас також має метод `Draw()`, який:

1. Прив'язує відповідні буфери.
2. Налаштовує вказівники атрибутів (`vertexAttribPointer`).
3. Викликає `gl.drawElements` для відображення сітки трикутників.

Ініціалізація шейдерної програми відбувається у функції `initGL()`. На цьому етапі створюється програма, компілюються вершинний та фрагментний шейдери, перевіряється коректність компіляції та споріднення. Після успішної побудови програми зчитуються всі потрібні локації атрибутів (позиція, нормаль, тангент, текстурна координата) та юніформів (матриці перетворень, параметри освітлення, сэмплери текстур, параметри масштабування текстури). Тут же задаються початкові значення складових моделі Фонга, тобто коефіцієнтів `ambient`, `diffuse` та `specular`, а також ступеня блиску (`shininess`). Текстури прив'язуються до конкретних текстурних юнітів, а до відповідних юніформів передаються індекси цих юнітів, щоб фрагментний шейдер міг коректно виконати вибірку з трьох різних текстур.

Обробка клавіатури для маніпуляцій з текстурою відбувається за допомогою функції `handleKeyDown`, що змінює положення точки на текстурі та масштаб текстури:

- A / D – зсув точки ліворуч/праворуч по осі  $u$  (горизонтальний напрям у текстурі).
- W / S – зсув точки вгору/вниз по осі  $v$ .
- Q / E – збільшення/зменшення масштабу текстури.

Після кожної зміни значення обмежуються (`clamp`) до критичних інтервалів, щоб уникнути виходу за межі текстури або надто малого/великого масштабу.

Важливою частиною реалізації є побудова базису TBN у вершинному шейдері. Нормалі та тангенти обчислюються аналітично в JavaScript та передаються як атрибути до GPU. У шейдері вони нормалізуються та використовуються для побудови бінормалі через векторний добуток нормалі та тангента. Такий підхід реалізує варіант Gram–Schmidt з пріоритетом дотичної, адже дотичний вектор вважається базовим, а нормаль коригується. Отримана матриця TBN застосовується для перетворення нормалі з `normal` з простору дотичної в простір споглядання, де вже реалізується модель освітлення Фонга з урахуванням рухомого джерела світла.

У фрагментному шейдері вся підготовлена на попередніх етапах інформація використовується для обчислення кінцевого кольору кожного пікселя за моделлю Фонга з використанням `normal` map. Спочатку з трьох текстур зчитуються значення: дифузна карта задає базовий колір поверхні, спекулярна карта визначає інтенсивність та форму блискучих відблисків, а `normal` map містить збурену нормаль у просторі дотичної. Нормаль із `normal` map декодується з діапазону  $[0;1]$  у вектор з компонентами в діапазоні  $[-1;1]$ , після чого перетворюється з простору дотичної в простір глядача за допомогою матриці TBN, сформованої у вершинному шейдері. Далі обчислюються нормалізовані напрямки на джерело світла та на спостерігача, визначається дифузна складова як скалярний добуток нормалі та напрямку на



світло. У підсумку дифузна й дзеркальна складові додатково модулюються значеннями з дифузної та спекулярної текстур, а отримана сума записується у вихідний колір фрагмента. Оскільки фрагментний шейдер працює з текстурними координатами, які вже масштабовані та зсунуті навколо вибраної точки у вершинному шейдері, користувач одразу бачить результат інтерактивного керування масштабом і положенням текстури на готовому зображенні.

#### 4. Інструкція користувача

Після відкриття файлу `index.html` у браузері користувач бачить головне вікно програми: по центру розташовано полотно з тривимірною поверхнею обертання, ліворуч — панель керування з повзунками та кнопками.

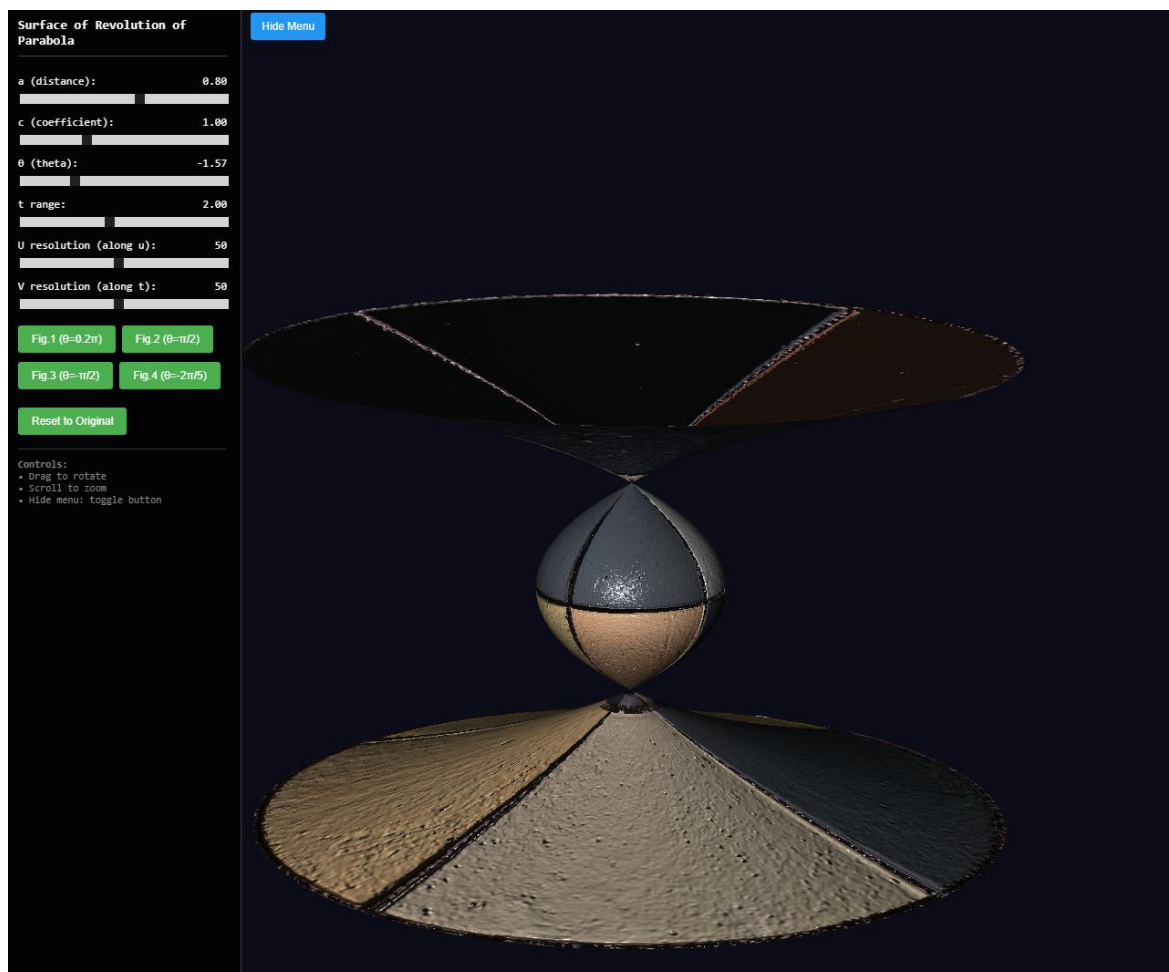


Рисунок 4.1 – Загальний вигляд застосунку.

Керування камерою здійснюється мишею. Перетягування із затиснутою лівою кнопкою обертає поверхню у просторі, дозволяючи оглянути її з різних ракурсів. Прокручування коліщатка миші відповідає за масштаб: прокрутка вперед наближає камеру, а прокрутка назад віддаляє. Це дає можливість спочатку виставити зручний ракурс для подальших експериментів із текстурами та параметрами моделі.

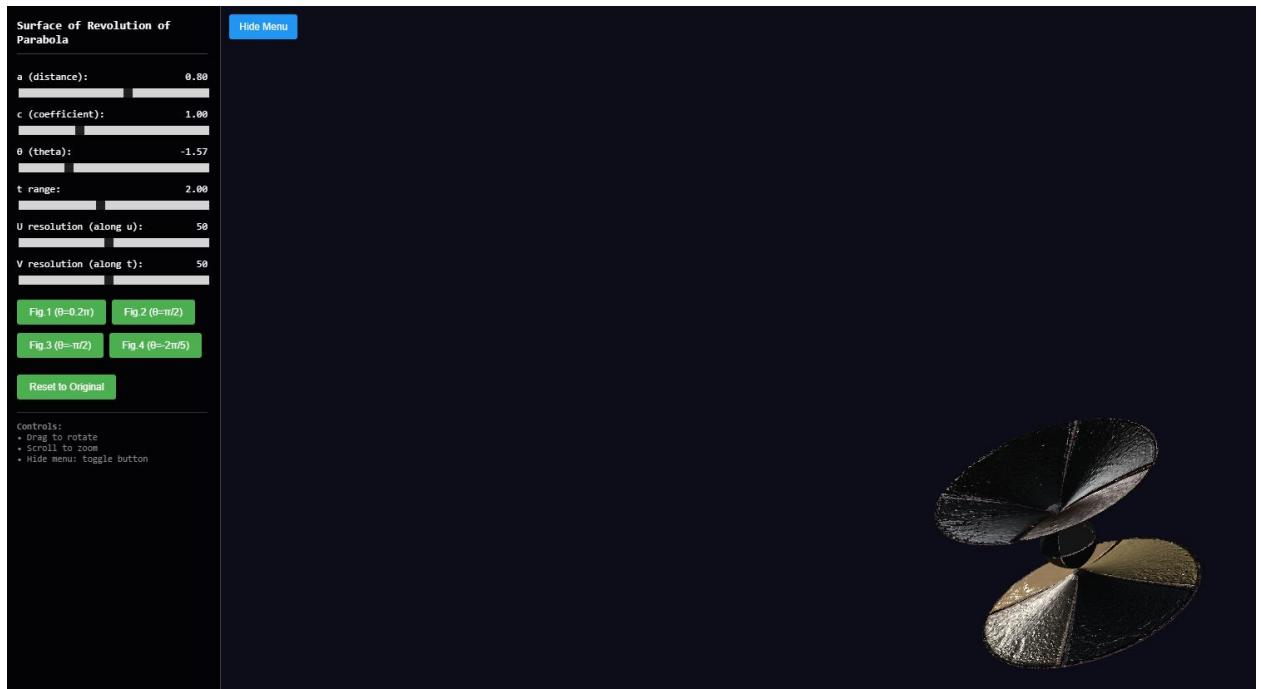


Рисунок 4.2 – Демонстрація обертання та масштабування поверхні мишею.

Панель керування дозволяє змінювати геометрію поверхні. Повзунки для параметрів  $a$ ,  $c$  та відповідають за форму параболі, що обертається, та її нахил. Повзунок діапазону  $t$  обрізає поверхню по висоті, а параметри дискретизації за напрямками  $u$  і  $v$  змінюють густоту сітки трикутників. Кнопки-пресети автоматично встановлюють комбінації параметрів, що відповідають окремим фігурам із завдання, а кнопка скидання повертає модель до початкового стану.

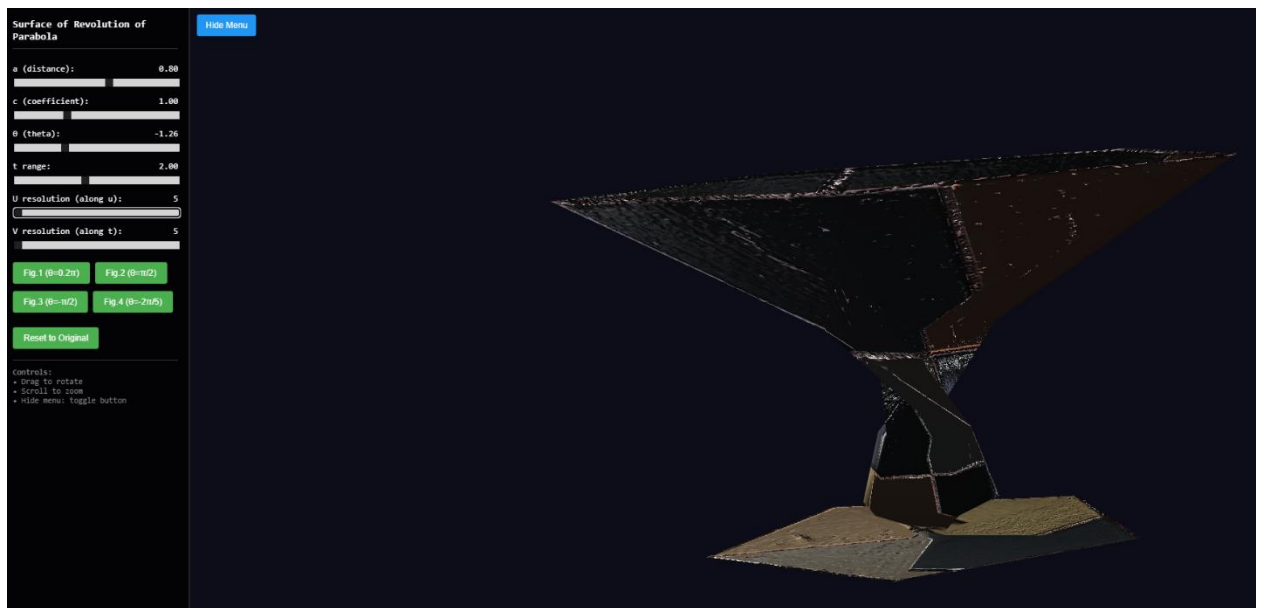


Рисунок 4.3 – Приклад зміни форми поверхні за допомогою повзунків та пресетів.

Окремий блок взаємодії стосується роботи з текстурою. Масштабування та зміщення текстури виконується з клавіатури. Після того, як вікно браузера активне, клавіші A і D зсувають опорну точку по горизонтальній координаті  $u$ , а W і S — по вертикальній координаті  $v$ . Таким чином, користувач може «пересунути» центр масштабування по поверхні, вибираючи ділянку, відносно якої текстура буде розтягуватись або стискатися. Клавіші Q та E змінюють масштаб: одна збільшує його, інша зменшує.

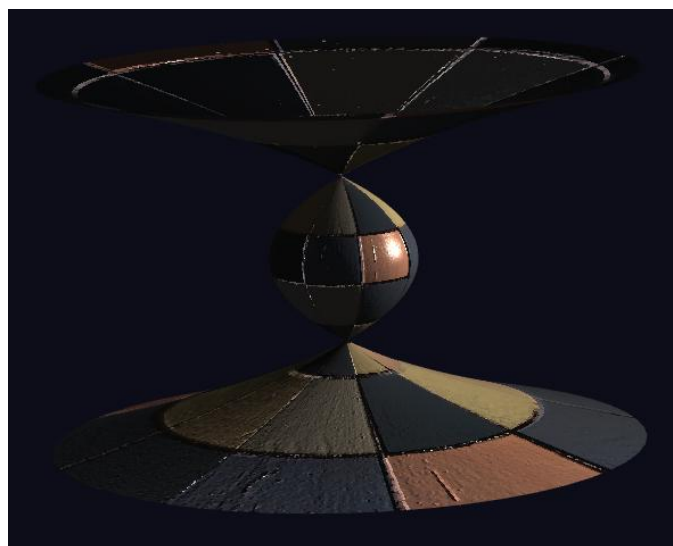


Рисунок 4.4 – Зміщення опорної точки текстури по поверхні

Під час зміни масштабу та положення опорної точки користувач спостерігає, як шаблон текстури наближається або віддаляється на поверхні, а дрібні деталі рельєфу, задані normal map, зберігаються й далі коректно підсвічуються. Джерело світла рухається по колу навколо поверхні, тому навіть при фіксованих параметрах текстури відблиски та тіні постійно змінюються, що добре демонструє поєднання трьох текстур дифузної, спекулярної та normal-map в одній моделі освітлення.

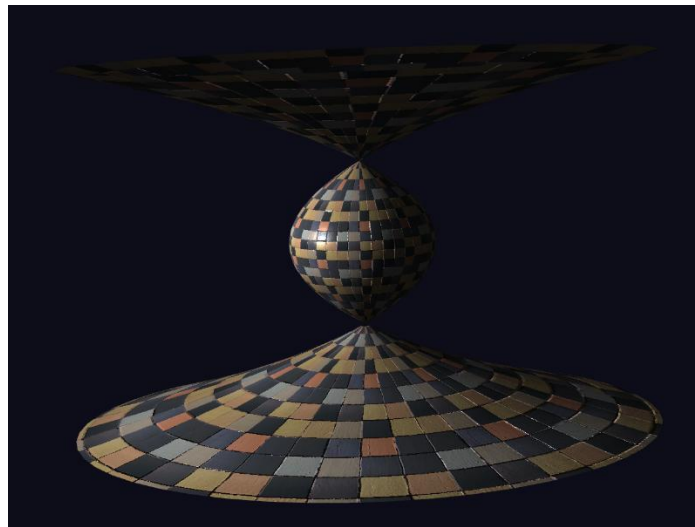


Рисунок 4.5 – Приклад малого масштабу текстури при одному й тому самому положенні точки.

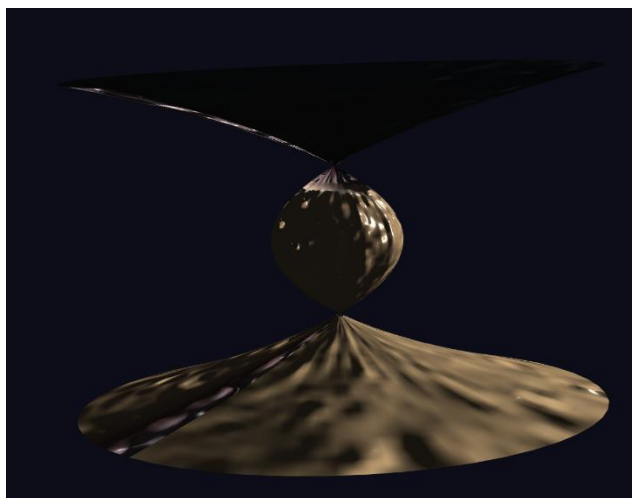


Рисунок 4.6 – Приклад великого масштабу текстури при одному й тому самому положенні точки.

У цьому розділі показано, що розроблений інтерфейс користувача забезпечує інтуїтивне дослідження геометрії поверхні та поведінки текстур. Керування мишею відповідає за навігацію у тривимірному просторі, а повзунки за зміну форми й масштабу.

Також продемонстровано інтерактивну роботу з UV-координатами, включно з переміщенням опорної точки та масштабуванням текстури, що наочно ілюструє вплив normal map і рухомого джерела світла. Застосунок слугує наочним інструментом для вивчення текстуризації та освітлення в WebGL.

## **5. Висновок**

У ході виконання роботи було розроблено інтерактивний WebGL-застосунок для візуалізації поверхні обертання параболи з керуванням геометричними параметрами, текстурним відображенням та освітленням. Реалізовано генерацію трикутної вершинної сітки на основі аналітичного задання поверхні, обчислення нормалей і тангентів, формування індексованих VBO, а також модель освітлення Фонга з використанням дифузної, спекулярної текстур і normal map, включно з побудовою TBN-базису та по компонентним розрахунком освітлення у фрагментному шейдері.

Практичним результатом є зручний інтерфейс користувача з інтерактивною навігацією сценою та зміною параметрів геометрії, доповнений механізмом інтерактивного текстурного масштабування навколо довільно вибраної точки в UV-просторі, яка може переміщуватися з клавіатури. Усі вимоги завдання виконано, а розроблений застосунок може використовуватися як наочний навчальний інструмент для вивчення шейдингу, текстуровання, normal mapping та освітлення в WebGL.

## 6. Зразки програмного коду

Нижче наведено фрагменти коду, які демонструють реалізацію масштабування текстури та керування точкою.

### Фрагменти main.js

```
let texPivot = { u: 0.5, v: 0.5 };
```

```
let texScale = 1.0;
```

```
/* Initialize the WebGL context. Called from init() */
```

```
function initGL() {
```

```
    let prog = createProgram(gl, vertexShaderSource, fragmentShaderSource);
```

```
    shProgram = new ShaderProgram('Basic', prog);
```

```
    shProgram.Use();
```

```
    // attributes
```

```
    shProgram.iAttribVertex = gl.getAttribLocation(prog, "vertex");
```

```
    shProgram.iAttribNormal = gl.getAttribLocation(prog, "normal");
```

```
    shProgram.iAttribTangent = gl.getAttribLocation(prog, "tangent");
```

```
    shProgram.iAttribTexCoord = gl.getAttribLocation(prog, "texCoord");
```

```
    // matrices
```

```
    shProgram.iModelViewProjectionMatrix = gl.getUniformLocation(prog,  
"ModelViewProjectionMatrix");
```

```
    shProgram.iModelViewMatrix          = gl.getUniformLocation(prog,  
"ModelViewMatrix");
```

```
    shProgram.iNormalMatrix             = gl.getUniformLocation(prog,  
"NormalMatrix");
```

```

// lighting
shProgram.iLightPosition      =      gl.getUniformLocation(prog,
"uLightPosition");

shProgram.iAmbient = gl.getUniformLocation(prog, "uAmbient");
shProgram.iDiffuse = gl.getUniformLocation(prog, "uDiffuse");
shProgram.iSpecular = gl.getUniformLocation(prog, "uSpecular");
shProgram.iShininess = gl.getUniformLocation(prog, "uShininess");


shProgram.iDiffuseMap = gl.getUniformLocation(prog, "uDiffuseMap");
shProgram.iSpecularMap      =      gl.getUniformLocation(prog,
"uSpecularMap");

shProgram.iNormalMap = gl.getUniformLocation(prog, "uNormalMap");


// texture scaling uniforms
shProgram.iTexPivot = gl.getUniformLocation(prog, "uTexPivot");
shProgram.iTexScale = gl.getUniformLocation(prog, "uTexScale");


gl.uniform3fv(shProgram.iAmbient, [0.1, 0.1, 0.1]);
gl.uniform3fv(shProgram.iDiffuse, [0.8, 0.8, 0.8]);
gl.uniform3fv(shProgram.iSpecular, [1.0, 1.0, 1.0]);
gl.uniform1f (shProgram.iShininess, 32.0);


gl.uniform1i(shProgram.iDiffuseMap, 0);
gl.uniform1i(shProgram.iSpecularMap, 1);
gl.uniform1i(shProgram.iNormalMap, 2);


// initial UV transform
gl.uniform2f(shProgram.iTexPivot, texPivot.u, texPivot.v);
gl.uniform1f(shProgram.iTexScale, texScale);

```

```

surface = new Model('Surface');
surface.BufferData(CreateSurfaceData());

diffuseTexture = createTexture("textures/diffuse.png");
specularTexture = createTexture("textures/specular.png");
normalTexture = createTexture("textures/normal.png");

gl.enable(gl.DEPTH_TEST);
}

```

### **Обробник клавіш:**

```

function handleKeyDown(e) {
    if (!gl || !shProgram) return;

    const step = 0.02;
    const scaleUp = 1.1;
    const scaleDown = 1.0 / scaleUp;

    switch (e.key.toLowerCase()) {
        case 'a':
            texPivot.u -= step;
            break;
        case 'd':
            texPivot.u += step;
            break;
        case 'w':
            texPivot.v += step;
            break;
        case 's':
            texPivot.v -= step;
            break;

        case 'q':
            texScale *= scaleUp;
            break;
        case 'e':
            texScale *= scaleDown;
            break;
        default:
            return;
    }
}

```



```

    }

    texPivot.u = Math.max(0.0, Math.min(1.0, texPivot.u));
    texPivot.v = Math.max(0.0, Math.min(1.0, texPivot.v));

    texScale = Math.max(0.1, Math.min(10.0, texScale));

    gl.uniform2f(shProgram.iTexPivot, texPivot.u, texPivot.v);
    gl.uniform1f(shProgram.iTexScale, texScale);
}

```

## Вершинний та фрагментний шейдери

Вершинний шейдер (shader.gpu):

```

// Vertex shader
const vertexShaderSource = `
attribute vec3 vertex;
attribute vec3 normal;
attribute vec3 tangent;
attribute vec2 texCoord;

uniform mat4 ModelViewProjectionMatrix;
uniform mat4 ModelViewMatrix;
uniform mat4 NormalMatrix;

uniform vec2 uTexPivot;
uniform float uTexScale;

varying vec3 vPosition;
varying vec2 vTexCoord;
varying vec3 vT;
varying vec3 vB;
varying vec3 vN;

void main() {
    vec4 posEye = ModelViewMatrix * vec4(vertex, 1.0);
    vPosition = posEye.xyz;

    vec2 centered = texCoord - uTexPivot;
    vec2 scaled = centered * uTexScale;
    vTexCoord = scaled + uTexPivot;

    vec3 N = normalize((NormalMatrix * vec4(normal, 0.0)).xyz);
    vec3 T = normalize((NormalMatrix * vec4(tangent, 0.0)).xyz);
}

```

```

    vec3 B = normalize(cross(N, T));

    vT = T;
    vB = B;
    vN = N;

    gl_Position = ModelViewProjectionMatrix * vec4(vertex, 1.0);
}
`;

```

Фрагментный шейдер:

// Fragment shader

```

const fragmentShaderSource = `
precision mediump float;

```

```

uniform vec3 uLightPosition;
uniform vec3 uAmbient;
uniform vec3 uDiffuse;
uniform vec3 uSpecular;
uniform float uShininess;

```

```

uniform sampler2D uDiffuseMap;
uniform sampler2D uSpecularMap;
uniform sampler2D uNormalMap;

```

```

varying vec3 vPosition;
varying vec2 vTexCoord;
varying vec3 vT;
varying vec3 vB;
varying vec3 vN;

```

```

void main() {
    vec3 texDiff = texture2D(uDiffuseMap, vTexCoord).rgb;
    vec3 texSpec = texture2D(uSpecularMap, vTexCoord).rgb;
    vec3 texNorm = texture2D(uNormalMap, vTexCoord).rgb;

    vec3 N_tan = normalize(texNorm * 2.0 - 1.0);

    mat3 TBN = mat3(vT, vB, vN);

    vec3 N = normalize(TBN * N_tan);

    vec3 L = normalize(uLightPosition - vPosition);
    vec3 V = normalize(vec3(0.0, 0.0, 1.0)); // as per hint

```

```
vec3 R = reflect(-L, N);

float lambert = max(dot(N, L), 0.0);
float spec    = pow(max(dot(R, V), 0.0), uShininess);

vec3 ambient = uAmbient * texDiff;
vec3 diffuse  = uDiffuse * texDiff * lambert;
vec3 specular = uSpecular * texSpec * spec;

vec3 color = ambient + diffuse + specular;

gl_FragColor = vec4(color, 1.0);
}
```